



Trabajo Practico Club Deportivo BD

Alumnos: Ian Gutierrez, Lukas Rilo

Materia: Administracion y Gestion de Bases de Datos

Curso: 5to 3ra Computacion Turno Noche

Año: 2026

Enunciado:

Situación problemática

El sistema debe administrar de manera eficiente los servicios prestados por el Club Deportivo.

Se han definido los siguientes requerimientos específicos y reglas de negocio:

Gestión de Personas: Se deben registrar los datos básicos (DNI, nombre, apellido, teléfono, email) de todas las personas. Se dividen en:

- o Socios: Poseen un número de carnet único y un estado de habilitación.
- o No Socios: Pagan un pase diario para acceder a actividades específicas.
- o Profesores/Personal: Cuentan con un legajo único y sueldo mensual.

Gestión de Actividades y Clases:

El club ofrece múltiples actividades (ej. Musculación, Spinning, Natación). Cada actividad puede tener distintas Clases planificadas en días y horarios específicos, localizadas en un salón determinado y dictadas por un Profesor a cargo.

Inscripción Dinámica a Actividades: * Los Socios y No Socios pueden inscribirse a las clases disponibles.

o Para que un Socio pueda registrarse o asistir a una actividad, el sistema debe validar obligatoriamente que su carnet esté activo y su cuota al día.

o Para un No Socio, la inscripción a la actividad requiere el registro previo del pago del pase diario correspondiente a esa fecha.

Administración de Cuotas, Pagos y Vencimientos:

o Emisión de Cuotas: Las cuotas de los socios se emiten mensualmente.

o Cálculo del Vencimiento Automatizado: El período de cobertura de la cuota es de un mes exacto. Al registrarse un pago, la fecha de vencimiento de la próxima cuota se calcula automáticamente para el mismo día del mes siguiente (o el primer día posterior si el mes siguiente es más corto).

o Inhabilitación Automática: Vencido el período de pago sin registrarse la nueva cuota, el sistema cambiará el estado del socio a Inhabilitado de forma automática, impidiendo su ingreso o inscripción a nuevas actividades. Para retomar, debe pagar la cuota pendiente; el nuevo plazo comenzará a correr a partir del día siguiente al pago.

Control del Plantel (Profesores):

o Se debe registrar la firma de una planilla de asistencia (fecha y hora de entrada) cada vez que se presentan al club.

o El cobro de su sueldo es mensual y se procesa de forma interna el último día hábil del mes.

Servicio Adicional de Nutrición y Salud:

o Los socios (únicamente si están al día) pueden solicitar turnos anticipados para el servicio de Nutrición.

o En la consulta, el nutricionista/médico confecciona una Ficha Médica donde se prescribe la "Carga de actividad física permitida (ej: liviana, moderada, intensa) y observaciones de diagnóstico. Las rutinas de ejercicios diseñadas por los profesores deben respetar estrictamente esta carga permitida.

Para el mismo se debe realizar

1. Modelo Entidad-Relación (MER)
 2. Diagrama de entidad relación
 3. Modelo Relacional (MR)
 4. Script SQL Completo (MySQL)
 5. Juego de Pruebas Operativo y consultas (Lo más completo posible)
- por ejemplo "Vencimientos Diarios", "estado de un socio actual", "listado de alumnos"

Modelo Entidad-Relación:

Persona:

Atributo	Tipo	Dominio	Restricciones		Clave	FK
dni	INT	0–99999999	NOT NULL	PK		
nombre	VARCHAR	50	NOT NULL			
apellido	VARCHAR	50	NOT NULL			
telefono	VARCHAR	20				
email	VARCHAR	100				

EstadoSocio:

Atributo	Tipo	Dominio	Restricciones		Clave	FK
id_estado	INT	Positivo		AUTO_INCREMENT, NOT NULL	PK	
descripcion	VARCHAR	20	NOT NULL, UNIQUE			

Socio:

Atributo	Tipo	Dominio	Restricciones		Clave	FK
nro_carnet	INT	Positivo	AUTO_INCREMENT, NOT NULL	PK		
dni	INT	0–99999999	NOT NULL, UNIQUE	FK		Persona(dni)
id_estado	INT	Positivo	NOT NULL	FK		EstadoSocio(id_estado)

NoSocio:

Atributo	Tipo	Dominio	Restricciones		Clave	FK
id_no_socio	INT	Positivo	AUTO_INCREMENT, NOT NULL	PK		
dni	INT	0–99999999	NOT NULL, UNIQUE	FK		Persona(dni)

Profesor:

Atributo	Tipo	Dominio	Restricciones	Clave	FK
legajo	INT	Positivo	AUTO_INCREMENT, NOT NULL	PK	
dni	INT	0-99999999	NOT NULL, UNIQUE	FK	Persona(dni)
sueldo	DECIMAL	10,2	NOT NULL		

Salon:

Atributo	Tipo	Dominio	Restricciones	Clave	FK
id_salon	INT	Positivo	AUTO_INCREMENT, NOT NULL	PK	
nombre	VARCHAR	50	NOT NULL		
capacidad	INT	Positivo	NOT NULL		

Actividad:

Atributo	Tipo	Dominio	Restricciones	Clave	FK
id_actividad	INT	Positivo	AUTO_INCREMENT, NOT NULL	PK	
nombre	VARCHAR	50	NOT NULL		
descripcion	VARCHAR	100			

DiaSemana:

Atributo	Tipo	Dominio	Restricciones	Clave	FK
id_dia	INT	Positivo	AUTO_INCREMENT, NOT NULL	PK	
nombre	VARCHAR	15	NOT NULL, UNIQUE		

Clase:

Atributo	Tipo	Dominio	Restricciones	Clave	FK
id_clase	INT	Positivo	AUTO_INCREMENT, NOT NULL	PK	
id_actividad	INT	Positivo	NOT NULL	FK	Actividad(id_actividad)
legajo	INT	Positivo	NOT NULL	FK	Profesor(legajo)
id_salon	INT	Positivo	NOT NULL	FK	Salon(id_salon)
id_dia	INT	Positivo	NOT NULL	FK	DiaSemana(id_dia)
horario	TIME	HH-MM-SS	NOT NULL		

Inscripcion:

Atributo	Tipo	Dominio	Restricciones	Clave	FK
id_inscripcion	INT	Positivo	AUTO_INCREMENT, NOT NULL	PK	
id_clase	INT	Positivo	NOT NULL	FK	Clase(id_clase)
nro_carnet	INT	Positivo	NULL	FK	Socio(nro_carnet)
id_no_socio	INT	Positivo	NULL	FK	NoSocio(id_no_socio)
fecha	DATE	Fecha válida	NOT NULL		

Cuota:

Atributo	Tipo	Dominio	Restricciones	Clave	FK
id_cuota	INT	Positivo	AUTO_INCREMENT, NOT NULL	PK	
nro_carnet	INT	Positivo	NOT NULL	FK	Socio(nro_carnet)
fecha_emision	DATE	Fecha válida	NOT NULL		
fecha_vencimiento	DATE	Fecha válida	NOT NULL		
importe	DECIMAL	10,2	NOT NULL		
pagada	BOOLEAN	TRUE/FALSE	DEFAULT FALSE		

PaseDiario:

Atributo	Tipo	Dominio	Restricciones	Clave	FK
id_pase	INT	Positivo	AUTO_INCREMENT, NOT NULL	PK	
id_no_socio	INT	Positivo	NOT NULL	FK	NoSocio(id_no_socio)
fecha	DATE	Fecha válida	NOT NULL		
importe	DECIMAL	10,2	NOT NULL		

AsistenciaProfesor:

Atributo	Tipo	Dominio	Restricciones	Clave	FK
id_asistencia	INT	Positivo	AUTO_INCREMENT, NOT NULL	PK	
legajo	INT	Positivo	NOT NULL	FK	Profesor(legajo)
fecha	DATE	Fecha válida	NOT NULL		
hora_entrada	TIME	HH-MM:SS	NOT NULL		

TurnoNutricion:

Atributo	Tipo	Dominio	Restricciones	Clave	FK
id_turno	INT	Positivo	AUTO_INCREMENT, NOT NULL	PK	
nro_carnet	INT	Positivo	NOT NULL	FK	Socio(nro_carnet)
fecha	DATE	Fecha válida	NOT NULL		
hora	TIME	HH-MM:SS	NOT NULL		

CargaActividad:

Atributo	Tipo	Dominio	Restricciones	Clave	FK
id_carga	INT	Positivo	AUTO_INCREMENT, NOT NULL	PK	
descripcion	VARCHAR	20	NOT NULL, UNIQUE		

FichaMedica:

Atributo	Tipo	Dominio	Restricciones	Clave	FK
id_ficha	INT	Positivo	AUTO_INCREMENT, NOT NULL	PK	
nro_carnet	INT	Positivo	NOT NULL	FK	Socio(nro_carnet)
id_carga	INT	Positivo	NOT NULL	FK	CargaActividad(id_carga)
observaciones	VARCHAR	255			

Diagrama Entidad-Relación:

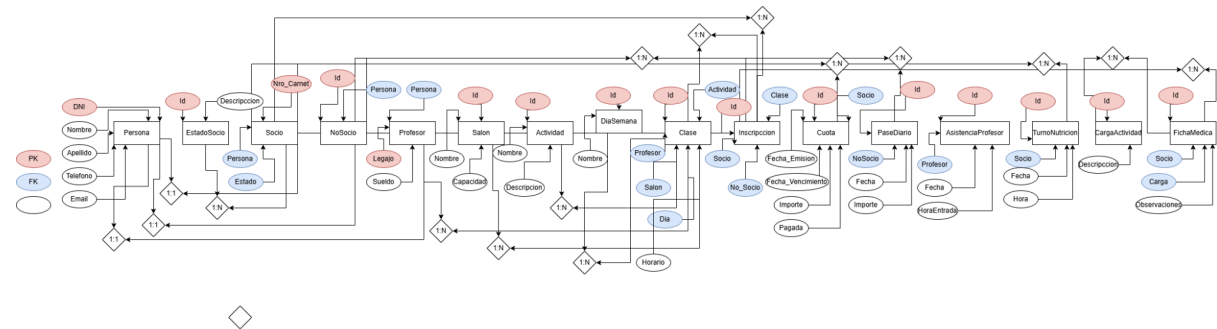
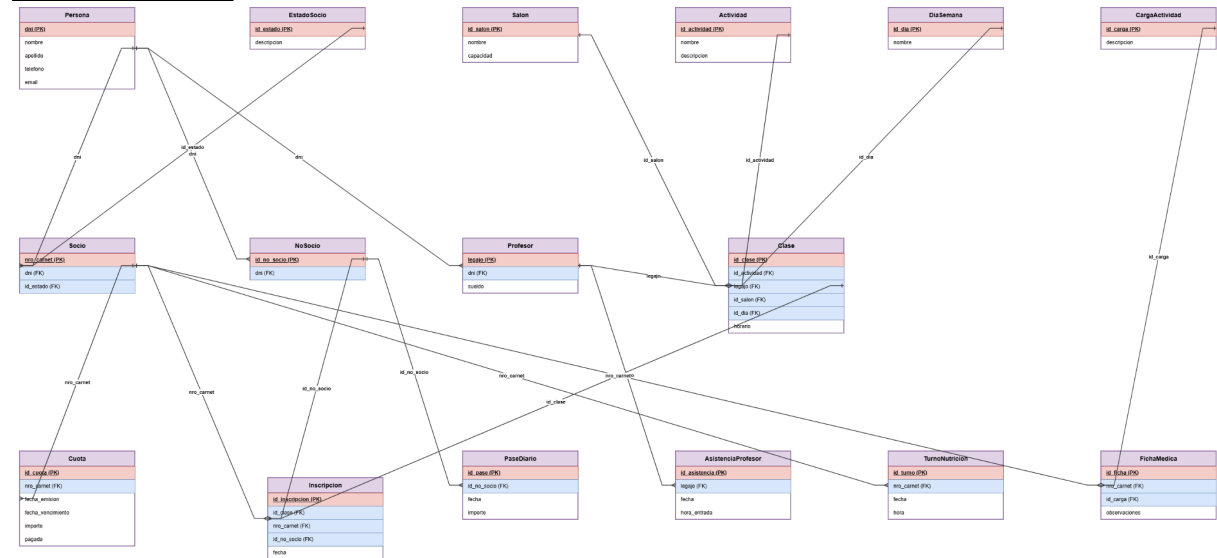


Diagrama Relacional:



MySQL completo:

Estructura:

```
DROP DATABASE IF EXISTS ClubDeportivo;  
CREATE DATABASE IF NOT EXISTS ClubDeportivo;  
USE ClubDeportivo;
```

```
CREATE TABLE Persona (  
    dni INT PRIMARY KEY,  
    nombre VARCHAR(50) NOT NULL,  
    apellido VARCHAR(50) NOT NULL,  
    telefono VARCHAR(20),  
    email VARCHAR(100)  
);
```

```
CREATE TABLE EstadoSocio (  
    id_estado INT AUTO_INCREMENT PRIMARY KEY,  
    descripcion VARCHAR(20) NOT NULL UNIQUE  
);
```

```
CREATE TABLE Socio (  
    nro_carnet INT AUTO_INCREMENT PRIMARY KEY,  
    dni INT NOT NULL UNIQUE,  
    id_estado INT NOT NULL,  
    FOREIGN KEY (dni)  
        REFERENCES Persona(dni)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT,  
    FOREIGN KEY (id_estado)  
        REFERENCES EstadoSocio(id_estado)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT  
);
```

```
CREATE TABLE NoSocio (  
    id_no_socio INT AUTO_INCREMENT PRIMARY KEY,  
    dni INT NOT NULL UNIQUE,  
    FOREIGN KEY (dni)  
        REFERENCES Persona(dni)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT  
);
```

```
CREATE TABLE Profesor (  
    legajo INT AUTO_INCREMENT PRIMARY KEY,  
    dni INT NOT NULL UNIQUE,  
    sueldo DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (dni)  
        REFERENCES Persona(dni)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT  
);
```

```
CREATE TABLE Salon (  
    id_salon INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(50) NOT NULL,  
    capacidad INT NOT NULL  
);
```

```
CREATE TABLE Actividad (  
    id_actividad INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(50) NOT NULL,  
    descripcion VARCHAR(100)  
);
```

```
CREATE TABLE DiaSemana (  
    id_dia INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(15) NOT NULL UNIQUE  
);
```

```
CREATE TABLE Clase (  
    id_clase INT AUTO_INCREMENT PRIMARY KEY,  
    id_actividad INT NOT NULL,  
    legajo INT NOT NULL,  
    id_salon INT NOT NULL,  
    id_dia INT NOT NULL,  
    horario TIME NOT NULL,  
    FOREIGN KEY (id_actividad)  
        REFERENCES Actividad(id_actividad)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT,  
    FOREIGN KEY (legajo)  
        REFERENCES Profesor(legajo)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT,  
    FOREIGN KEY (id_salon)  
        REFERENCES Salon(id_salon)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT,  
    FOREIGN KEY (id_dia)  
        REFERENCES DiaSemana(id_dia)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT  
);
```

```

CREATE TABLE Inscripcion (
  id_inscripcion INT AUTO_INCREMENT PRIMARY KEY,
  id_clase INT NOT NULL,
  nro_carnet INT NULL,
  id_no_socio INT NULL,
  fecha DATE NOT NULL,
  FOREIGN KEY (id_clase)
    REFERENCES Clase(id_clase)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  FOREIGN KEY (nro_carnet)
    REFERENCES Socio(nro_carnet)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  FOREIGN KEY (id_no_socio)
    REFERENCES NoSocio(id_no_socio)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
);

```

```

CREATE TABLE Cuota (
  id_cuota INT AUTO_INCREMENT PRIMARY KEY,
  nro_carnet INT NOT NULL,
  fecha_emision DATE NOT NULL,
  fecha_vencimiento DATE NOT NULL,
  importe DECIMAL(10,2) NOT NULL,
  pagada BOOLEAN DEFAULT FALSE,
  FOREIGN KEY (nro_carnet)
    REFERENCES Socio(nro_carnet)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
);

```

```

CREATE TABLE PaseDiario (
  id_pase INT AUTO_INCREMENT PRIMARY KEY,
  id_no_socio INT NOT NULL,
  fecha DATE NOT NULL,
  importe DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (id_no_socio)
    REFERENCES NoSocio(id_no_socio)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
);

```

```

CREATE TABLE AsistenciaProfesor (
  id_asistencia INT AUTO_INCREMENT PRIMARY KEY,
  legajo INT NOT NULL,
  fecha DATE NOT NULL,
  hora_entrada TIME NOT NULL,
  FOREIGN KEY (legajo)
    REFERENCES Profesor(legajo)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
);

```

```

CREATE TABLE TurnoNutricion (
  id_turno INT AUTO_INCREMENT PRIMARY KEY,
  nro_carnet INT NOT NULL,
  fecha DATE NOT NULL,
  hora TIME NOT NULL,
  FOREIGN KEY (nro_carnet)
    REFERENCES Socio(nro_carnet)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
);

```

```

CREATE TABLE CargaActividad (
  id_carga INT AUTO_INCREMENT PRIMARY KEY,
  descripcion VARCHAR(20) NOT NULL UNIQUE
);

```

```

CREATE TABLE FichaMedica (
  id_ficha INT AUTO_INCREMENT PRIMARY KEY,
  nro_carnet INT NOT NULL,
  id_carga INT NOT NULL,
  observaciones VARCHAR(255),
  FOREIGN KEY (nro_carnet)
    REFERENCES Socio(nro_carnet)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  FOREIGN KEY (id_carga)
    REFERENCES CargaActividad(id_carga)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
);

```

Queries(Juego de pruebas):

1) Listado de todas las personas
 SELECT * FROM Persona;

2) Listado de socios con su estado

```
SELECT
    s.nro_carnet,
    p.nombre,
    p.apellido,
    es.descripcion AS estado
FROM Socio s
JOIN Persona p ON s.dni = p.dni
JOIN EstadoSocio es ON s.id_estado = es.id_estado;
```

3) Estado actual de un socio

```
SELECT
    p.nombre,
    p.apellido,
    es.descripcion AS estado
FROM Socio s
JOIN Persona p ON s.dni = p.dni
JOIN EstadoSocio es ON s.id_estado = es.id_estado
WHERE s.nro_carnet = 1;
```

4) Vencimientos diarios

```
SELECT
    c.id_cuota,
    p.nombre,
    p.apellido,
    c.fecha_vencimiento
FROM Cuota c
JOIN Socio s ON c.nro_carnet = s.nro_carnet
JOIN Persona p ON s.dni = p.dni
WHERE c.fecha_vencimiento = CURDATE();
```

5) Cuotas pagadas

```
SELECT *
FROM Cuota
WHERE pagada = TRUE;
```

6) Cuotas pendientes

```
SELECT *
FROM Cuota
WHERE pagada = FALSE;
```

7) Listado de profesores

```
SELECT
    pr.legajo,
    pe.nombre,
    pe.apellido,
    pr.sueldo
FROM Profesor pr
JOIN Persona pe ON pr.dni = pe.dni;
```

8) Asistencia de profesores

```
SELECT
    pe.nombre,
    pe.apellido,
    ap.fecha,
    ap.hora_entrada
FROM AsistenciaProfesor ap
JOIN Profesor pr ON ap.legajo = pr.legajo
JOIN Persona pe ON pr.dni = pe.dni;
```

9) Listado de actividades

```
SELECT * FROM Actividad;
```

10) Clases programadas

```
SELECT
    c.id_clase,
    a.nombre AS actividad,
    ds.nombre AS dia,
    c.horario,
    sa.nombre AS salon
FROM Clase c
JOIN Actividad a ON c.id_actividad = a.id_actividad
JOIN DiaSemana ds ON c.id_dia = ds.id_dia
JOIN Salon sa ON c.id_salon = sa.id_salon;
```

11) Listado de alumnos inscriptos

```
SELECT
    i.id_inscripcion,
    a.nombre AS actividad,
    c.horario,
    COALESCE(ps.nombre, pns.nombre) AS nombre,
    COALESCE(ps.apellido, pns.apellido) AS apellido
FROM Inscripcion i
JOIN Clase c ON i.id_clase = c.id_clase
JOIN Actividad a ON c.id_actividad = a.id_actividad
LEFT JOIN Socio s ON i.nro_carnet = s.nro_carnet
LEFT JOIN Persona ps ON s.dni = ps.dni
LEFT JOIN NoSocio ns ON i.id_no_socio = ns.id_no_socio
LEFT JOIN Persona pns ON ns.dni = pns.dni;
```

12) No socios con pase diario

```
SELECT
    p.nombre,
    p.apellido,
    pd.fecha,
    pd.importe
FROM PaseDiario pd
JOIN NoSocio ns ON pd.id_no_socio = ns.id_no_socio
JOIN Persona p ON ns.dni = p.dni;
```

13) Turnos de nutrición

```
SELECT
    p.nombre,
    p.apellido,
    tn.fecha,
    tn.hora
FROM TurnoNutricion tn
JOIN Socio s ON tn.nro_carnet = s.nro_carnet
JOIN Persona p ON s.dni = p.dni;
```

14) Fichas médicas

```
SELECT
    p.nombre,
    p.apellido,
    ca.descripcion AS carga_permitida,
    fm.observaciones
FROM FichaMedica fm
JOIN Socio s ON fm.nro_carnet = s.nro_carnet
JOIN Persona p ON s.dni = p.dni
JOIN CargaActividad ca ON fm.id_carga = ca.id_carga;
```

15) Cantidad de socios

```
SELECT COUNT(*) AS total_socios
FROM Socio;
```

16) Cantidad de profesores

```
SELECT COUNT(*) AS total_profesores
FROM Profesor;
```

17) Cantidad de actividades

```
SELECT COUNT(*) AS total_actividades
FROM Actividad;
```

18) Cantidad de clases por actividad

```
SELECT
    a.nombre,
    COUNT(*) AS cantidad_clases
FROM Clase c
JOIN Actividad a ON c.id_actividad = a.id_actividad
GROUP BY a.nombre;
```

19) Socios habilitados

```
SELECT
    p.nombre,
    p.apellido
FROM Socio s
JOIN Persona p ON s.dni = p.dni
JOIN EstadoSocio es ON s.id_estado = es.id_estado
WHERE es.descripcion = 'Habilitado';
```

20) Clases con su profesor

SELECT

a.nombre AS actividad,

pe.nombre,

pe.apellido,

ds.nombre AS dia,

c.horario

FROM Clase c

JOIN Actividad a ON c.id_actividad = a.id_actividad

JOIN Profesor pr ON c.legajo = pr.legajo

JOIN Persona pe ON pr.dni = pe.dni

JOIN DiaSemana ds ON c.id_dia = ds.id_dia;